ApexFi V2

Model Evaluation Report

How far to trust the fraud engine, and what changed since V1

Real-time UPI/IMPS-style fraud detection with explainable AI

Prepared by **Karthikeyan R**
Amrita Vishwa Vidyapeetham, Bengaluru Campus

Data: combined Golden layer (IEEE-CIS 590,540 rows $\times$ 442 columns; DGraph-Fin 3,700,550 nodes, 4,300,999 edges)

Every number in this report is read from a result file produced by the scripts listed in Section 14. Nothing is estimated by hand. Assumptions are labelled as assumptions.

October 27, 2026

Contents

1 Decision and key results **3**

2 V1 re-run and what V2 changed **3**

2.1 What V2 adds . . . 4

2.2 Data-quality problems found in V1 and the Gold layer . . . 4

3 Combined Golden layer **5**

4 Data analysis: univariate, bivariate, multivariate **5**

4.1 Univariate . . . 5

4.2 Bivariate (each feature against fraud) . . . 5

4.3 Multivariate . . . 6

5 Evaluation design **7**

6 Benchmark of 25 classifiers **8**

6.1 Time-ordered split (main result) . . . 9

6.2 Random split compared with time-ordered . . . 10

6.3 Reading the benchmark . . . 11

7 Features, imbalance and tuning **12**

7.1 Feature configurations . . . 12

7.2 Feature selection: 8 methods . . . 12

7.3 Class imbalance: SMOTE and others . . . 13

7.4 Hyper-parameter search . . . 13

8 Final comparison and finance metrics **14**

8.1 Finance and risk metrics (test block) . . . 15

8.2 What drives the model (SHAP) . . . 16

9 DGraph-Fin, corrected **16**

9.1 The leak, shown by ablation . . . 16

9.2 V2 feature sets (no timestamp) . . . 16

9.3 25-model benchmark on DGraph-Fin . . . 17

10 SDG Lab: manual and engine-driven **17**

10.1 Engine replay on the real test block . . . 18

10.2 Scaling up (all SCENARIO) . . . 18

11 Money and the friction assumption **19**

12 Limitations and honest notes **19**

13 Next steps **20**

14 Evidence and reproducibility **20**

1 Decision and key results

Production model confirmed: tuned LightGBM (449 features).
Test PR-AUC 0.524, ROC-AUC 0.907 on 147,635 future transactions (5,100 frauds). The closest rival, Voting of tuned LightGBM + XGBoost + CatBoost, was only 0.71% better on validation. We required 1% to switch, so we keep the single model: it is simpler and fully explainable with TreeSHAP.

	V1 temporal model	V2 final model
Features used	7	449
Test PR-AUC (main score)	0.181	0.524
Test ROC-AUC	0.790	0.907
Precision / recall at own threshold	0.16 / 0.51	0.62 / 0.43
F1	0.237	0.510
KS statistic	0.455	0.654
Calibration error (ECE)	0.298	0.0076
Share of all transactions flagged (alert rate)	11.2%	2.4%

Same 147,635-transaction test block for both. V1 threshold 0.6 is the one V1 chose; V2 threshold 0.21 is chosen on a separate validation block (F1-optimal).

What the results tell us

- **V2 beats V1 by a wide margin.** PR-AUC rises by 0.343 (95% bootstrap interval 0.329–0.357); ROC-AUC by 0.117.
- **Most of the gain is simply using more Gold columns.** The 432 raw Gold columns alone score test PR-AUC 0.474; the full V2 set scores 0.476. The rolling and frequency features we engineered add almost nothing on this dataset (Section 7.1). V1 used 7 columns.
- **A random split flatters every model.** Best random-split PR-AUC is 0.703 against 0.488 on the time-ordered split (Section 6.2). Only the time-ordered number is a fair estimate of future use.
- **Algorithm choice matters less than expected.** The top 8 models (boosting and ensembles) sit within 3.4% of each other on test PR-AUC. Data and features limit accuracy more than the algorithm.
- **Tuning gives a small, real gain.** Optuna improved validation PR-AUC by 2.2%; the three search methods end close together (Section 7.4).
- **SMOTE is not needed.** Over three seeds it changes validation PR-AUC by 0.5%, inside run-to-run noise (Section 7.3).
- **The old DGraph-Fin result was a label leak.** V1 reported ROC-AUC 0.938. Removing one column (`node_timestamp`) gives ROC-AUC 0.726 and PR-AUC 0.026, in the range published for this dataset (Section 9).
- **Money depends on one assumption.** With V1's $1 friction cost, flagging every transaction beats any model. The break-even is about 1.35 dollars per false alarm. Above it the model wins (Section 11).

2 V1 re-run and what V2 changed

V1 is kept as the base. We re-ran its logic on the combined Golden layer (same split seed 42, same LightGBM settings) and compared with the stored V1 results.

Check	V1 stored	Re-run	Verdict
IEEE-CIS standard, LightGBM (ROC-AUC)	0.9733 (3-seed mean)	0.9724	Reproduced
IEEE-CIS standard (F1 at 0.5)	0.796	0.784	Reproduced (one seed)
Random split, 4 frequency features (ROC-AUC)	0.8463	0.8463	Exact
Time-ordered, frequency only	0.7691	0.7691	Exact
Time-ordered, rolling only	0.6273	0.6273	Exact
Time-ordered, frequency + rolling	0.7898	0.7918	Close (0.002 higher)
DGraph-Fin standard (ROC-AUC)	0.9377	0.9297	Reproduced, but see Section 9: it is a leak

All re-runs read the combined Gold store. The ROC-AUC difference of 0.002 in the last time-ordered row comes from tie handling inside the one-hour rolling window.

2.1 What V2 adds

1. One combined Golden store (Section 3).
2. Univariate, bivariate and multivariate analysis of every feature (Section 4).
3. A 25-model benchmark, including a baseline, bagging, boosting, voting and stacking (Section 6).
4. Feature configurations, 8 feature-selection methods, SMOTE and other rebalancing, and three hyper-parameter searches (Section 7).
5. Finance and risk metrics, not just F1 (Section 8).
6. A corrected DGraph-Fin evaluation (Section 9).
7. An SDG Lab with two tabs: manual calculator and engine-driven (Section 10).

2.2 Data-quality problems found in V1 and the Gold layer

- **`node_timestamp` (DGraph-Fin) is filled only for fraud accounts** (all 15,509 fraud nodes have it; all 1,210,092 normal nodes are empty). V1 replaced empty values by 0, so a non-zero value meant fraud. 8,024 of 15,509 fraud nodes have a non-zero value, which is why V1 recall was about 0.52 with precision about 0.99. V2 excludes the column.
- **Column M4 is 100% empty in the Gold layer** (0 of 590,540 values). In the original dataset M4 holds M0/M1/M2; the Silver step converts only `T`/`F` and turns everything else into NULL. The raw values cannot be seen in the files provided, so please confirm in Bronze with `select m4, count(*) from bronze.raw_ieee_cis group by 1`. Fixing it needs a Bronze-to-Gold re-run. This is not done in V2.
- **Three RBI rate columns** (`bank_rate`, `sbi_lending_rate`, `call_money_rate`) are joined to IEEE-CIS by fiscal year. They are constant or empty, mean nothing for US-dollar card data, and act as a time proxy. V2 removes them. V1 standard mode kept them.
- **V1 temporal mode used only 7 of the Gold columns.** This understated what the Gold layer can do (Section 7.1).
- **Class weighting with the full ratio hurts ranking.** V1's weighted recipe scores test PR-AUC 0.248 against 0.474 for the unweighted model.

3 Combined Golden layer

All Gold tables now live in one DuckDB store, `apexfi_gold.duckdb`. The datasets have different columns and meanings, so rows are **not** stacked into one training table. Instead: (1) each dataset keeps its own Gold table, (2) a catalogue table describes each one, and (3) a small `gold_unified_events` table holds only the shared columns (dataset, entity id, time, label, amount) for cross-dataset summaries. Models are trained per dataset with the same pipeline.

Dataset	Kind	Rows	Cols	Time range	Fraud rate
ieee_cis	tabular transactions	590,540	442	86400-15811131 s	3.50%
dgraph_fin	graph accounts (+4,300,999 edges)	3,700,550	25	0-793 days	1.27%
npci_monthly	context	12	3	monthly	—
npci_quarterly	context	4	3	quarterly	—
rbi_money_rates	context	18	7	fiscal year	—

DGraph-Fin fraud rate is over the 1,225,601 labelled nodes (the other 2,474,949 are unlabelled background accounts). The Ethereum side experiment is not part of the exported Gold files and is not re-run here.

Macro context from the Gold layer. The NPCI table gives monthly volume (crore) and value (lakh crore). Dividing value by volume gives an average UPI ticket of about `INR 1,600` in the last month shown (12.82 lakh crore over 782.95 crore payments). The IEEE-CIS average is $135. So dollar results here do not transfer to UPI; scale and ticket size differ by roughly an order of magnitude. The NPCI table does not store the year, so it is used only for this ratio.

4 Data analysis: univariate, bivariate, multivariate

Per-feature tables for **every** column are saved as CSV (see Section 14). The figures and numbers below summarise them. IEEE-CIS bivariate and multivariate steps use a fixed random sample of 200,000 rows (seed 7); the univariate table uses all 590,540 rows.

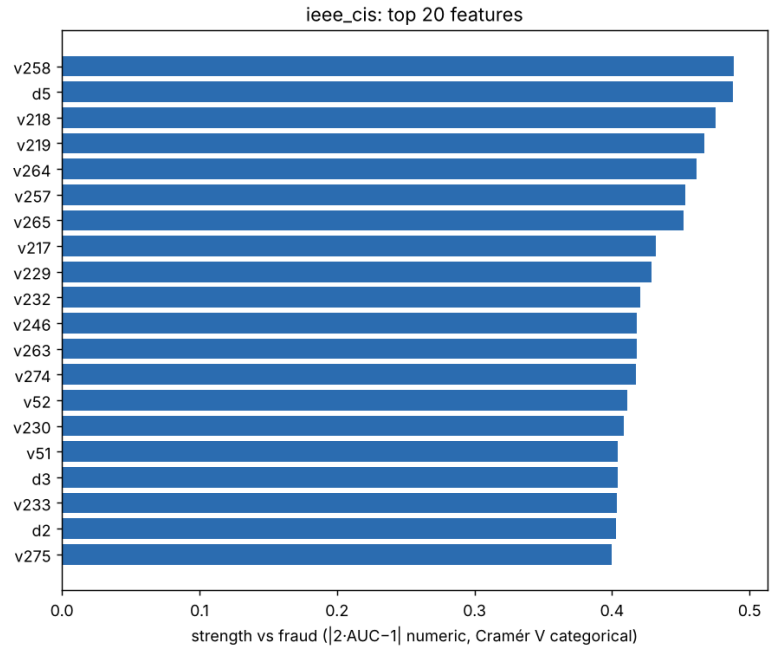
4.1 Univariate

Item	IEEE-CIS	DGraph-Fin (labelled)
Columns analysed	438	21
Numeric / categorical	415 / 23	21 / 0
Fraud rate	3.54%	1.27%
Constant columns	4	0
Near-constant (>99% one value)	30	0
Columns with >50% missing	210	1
Average missing per column	45.0%	4.7%

Per-column mean, spread, quantiles, skew, kurtosis, missing share and top categories are in the CSV files.

4.2 Bivariate (each feature against fraud)

Numeric features are scored by single-feature AUC, KS statistic, mutual information and fraud rate when missing versus present. Categorical features use Cramérś V and information value.



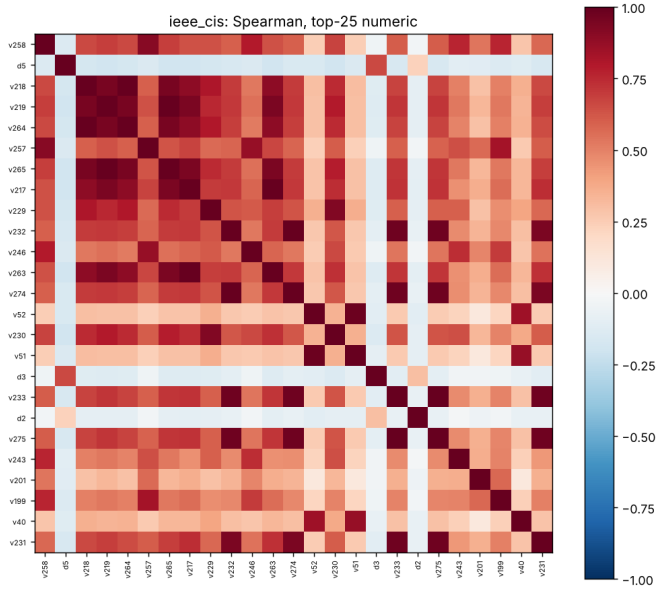
Top 20 IEEE-CIS features by single-feature strength. Many are Vesta V-columns that are missing for 78% of rows: fraud rate is 7.9% when present and 2.3% when missing, so *being missing* is itself a signal.

Strongest categorical signals: `deviceinfo` (Cramérś V 0.18), `r_emaildomain` (0.18, information value 0.65), browser version `id_31` (0.18) and product code (0.17). In DGraph-Fin the strongest single features are `out_degree`, `x2`, `x3`, `x6` and `x8`, all with AUC about 0.69–0.71 (below 0.5 in the raw direction, meaning higher values mean safer). **`node_timestamp`** shows fraud rate 0.0 when missing and 1.0 when present: the leak in one line.

4.3 Multivariate

Item	IEEE-CIS	DGraph-Fin
Numeric features used	410	21
Pairs with $\lvert\text{Spearman}\rvert > 0.9$	517	18
Redundant clusters (connected at 0.9)	78	4
Features inside a redundant cluster	261	13
Components for 90% variance (PCA)	>60 (cap 60)	8
Variance in first component	15.2%	49.7%

IEEE-CIS: 261 of 410 numeric features sit in 78 groups of near-duplicates (mostly V-columns). This is why feature selection can drop many columns with no loss (Section 7.2).

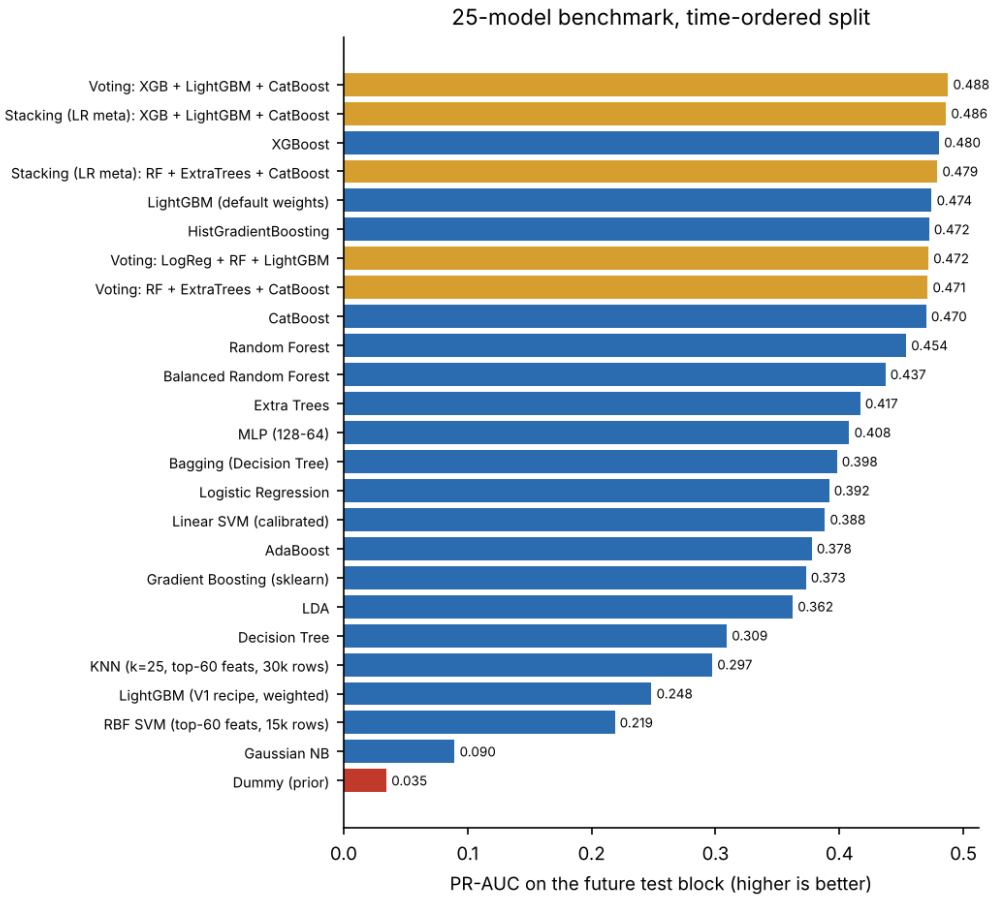


Spearman correlation among the 25 strongest IEEE-CIS numeric features: they move in blocks.

5 Evaluation design

Two kinds of test. *Time-ordered*: train on the first 75% of transactions by time, test on the last 25% (147,635 rows). The last 15% of the training window is a validation block used for early stopping, threshold choice and stacker weights. *Random*: stratified random 75/25 split of the same rows, shown only for comparison. **Fair and leak-safe.** Frequency maps are fitted on the training pool only. Rolling features use strictly earlier transactions. The graph-degree features of V1 standard mode are **not** used in time-ordered mode (edges use the whole period). The test block is scored once; winners are chosen on the validation block. **Training sample.** To make 25 models affordable on a 2-CPU machine, every model in the benchmark is fitted on the same random 100,000-row sample of the fit block (376,469 rows). Slow models are cut further: KNN 30,000 rows and RBF-SVM 15,000 rows, both on the 60 strongest features. The final production model is refitted on all 376,469 rows. Test rows are never sampled. **Metrics, and why.** Only 3.5% of rows are fraud, so accuracy and ROC-AUC alone mislead. The **main score is PR-AUC**. We also report ROC-AUC, F1, MCC, KS, Gini, Brier score, calibration error, lift, recall at fixed alert rate or false-positive rate, value-weighted recall, and score drift (PSI). Money is reported through a sensitivity grid because it depends on assumptions. **Switch rule (same as the DEM report).** A challenger replaces the incumbent only if it beats it by at least 1% (relative) on validation PR-AUC.

6 Benchmark of 25 classifiers



Test PR-AUC for all 25 models, time-ordered split. Red = baseline, amber = ensemble, blue = single models. Ordered by test score; the winner is chosen on validation.

6.1 Time-ordered split (main result)

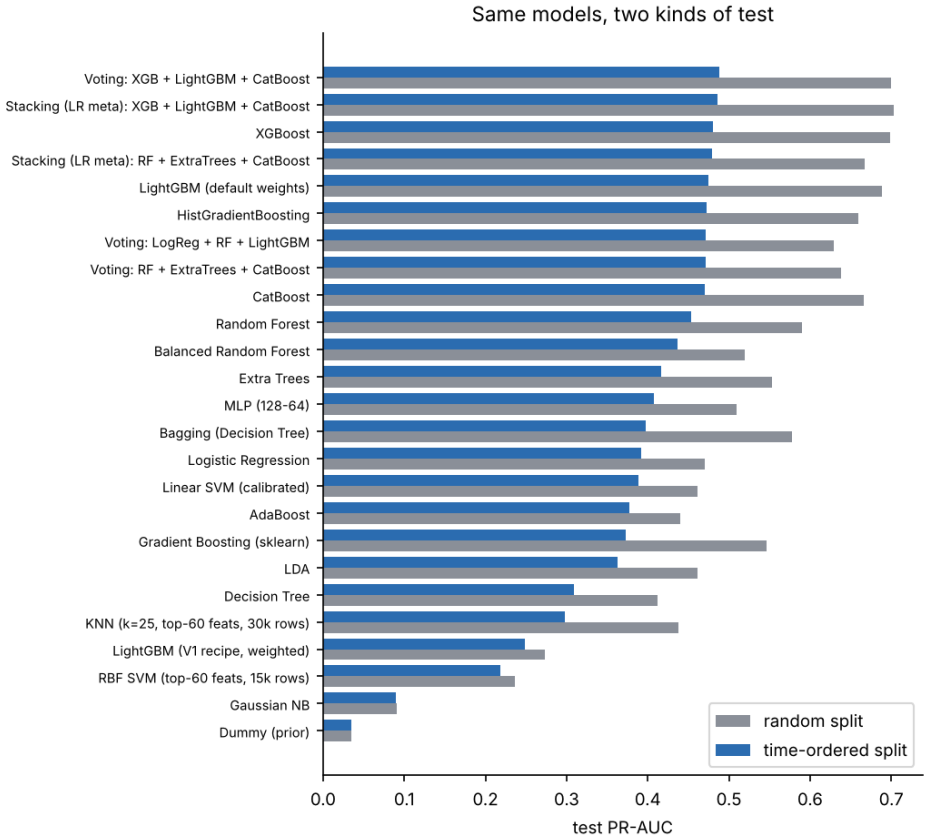
#	Model	Family	Val PR	Test PR	Test ROC	F1	Prec	Recall	MCC	KS
1	Voting: XGB + LightGBM + CatBoost	Ensemble	0.621	0.488	0.892	0.485	0.60	0.41	0.48	0.62
2	Stacking (LR meta): XGB + LightGBM + CatBoost	Ensemble	0.620	0.486	0.891	0.481	0.61	0.40	0.48	0.62
3	XGBoost	Boosting	0.615	0.480	0.887	0.477	0.58	0.41	0.47	0.62
4	Stacking (LR meta): RF + ExtraTrees + CatBoost	Ensemble	0.609	0.479	0.886	0.477	0.60	0.39	0.47	0.62
5	LightGBM (default weights)	Boosting	0.608	0.474	0.886	0.469	0.51	0.43	0.45	0.61
6	CatBoost	Boosting	0.601	0.470	0.880	0.471	0.65	0.37	0.48	0.60
7	Voting: RF + ExtraTrees + CatBoost	Ensemble	0.598	0.471	0.880	0.464	0.61	0.38	0.46	0.61
8	Voting: LogReg + RF + LightGBM	Ensemble	0.595	0.472	0.878	0.472	0.58	0.40	0.47	0.59
9	HistGradientBoosting	Boosting	0.592	0.472	0.887	0.472	0.59	0.40	0.47	0.62
10	Random Forest	Bagging	0.570	0.454	0.878	0.456	0.58	0.38	0.45	0.60
11	Bagging (Decision Tree)	Bagging	0.561	0.398	0.855	0.375	0.39	0.36	0.35	0.57
12	Balanced Random Forest	Bagging	0.551	0.437	0.874	0.442	0.57	0.36	0.44	0.59
13	Extra Trees	Bagging	0.536	0.417	0.862	0.424	0.56	0.34	0.42	0.58
14	Logistic Regression	Individual	0.535	0.392	0.854	0.412	0.48	0.36	0.40	0.55
15	Linear SVM (calibrated)	Individual	0.532	0.388	0.856	0.405	0.46	0.36	0.39	0.56
16	Gradient Boosting (sklearn)	Boosting	0.524	0.373	0.859	0.383	0.40	0.37	0.36	0.57
17	LDA	Individual	0.521	0.362	0.829	0.401	0.45	0.36	0.38	0.53
18	AdaBoost	Boosting	0.500	0.378	0.858	0.389	0.45	0.34	0.37	0.57
19	MLP (128-64)	Individual	0.499	0.408	0.819	0.435	0.55	0.36	0.43	0.52
20	KNN (k=25, top-60 feats, 30k rows)	Individual	0.437	0.297	0.769	0.304	0.36	0.27	0.29	0.44
21	Decision Tree	Individual	0.401	0.309	0.758	0.343	0.37	0.32	0.32	0.49
22	LightGBM (V1 recipe, weighted)	Boosting	0.382	0.248	0.828	0.311	0.34	0.29	0.29	0.52
23	RBF SVM (top-60 feats, 15k rows)	Individual	0.304	0.219	0.805	0.316	0.28	0.36	0.29	0.47
24	Gaussian NB	Individual	0.111	0.090	0.751	0.186	0.11	0.57	0.19	0.47
25	Dummy (prior)	Baseline	0.041	0.035	0.500	0.067	0.03	1.00	0.00	0.02

= rank on validation PR-AUC. F1, precision, recall, MCC and KS use the threshold that maximised F1 on the validation block. Model notes: KNN 30,000 rows and RBF-SVM 15,000 rows, 60 features each; Linear SVM is calibrated; the Dummy model predicts the training fraud rate.

6.2 Random split compared with time-ordered

Model	PR random	PR time-ordered	PR drop	ROC random	ROC time-ordered
Voting: XGB + LightGBM + CatBoost	0.700	0.488	30%	0.932	0.892
Stacking (LR meta): XGB + LightGBM + CatBoost	0.703	0.486	31%	0.934	0.891
XGBoost	0.699	0.480	31%	0.932	0.887
Stacking (LR meta): RF + ExtraTrees + CatBoost	0.668	0.479	28%	0.923	0.886
LightGBM (default weights)	0.688	0.474	31%	0.930	0.886
HistGradientBoosting	0.660	0.472	28%	0.922	0.887
Voting: LogReg + RF + LightGBM	0.629	0.472	25%	0.905	0.878
Voting: RF + ExtraTrees + CatBoost	0.638	0.471	26%	0.915	0.880
CatBoost	0.666	0.470	29%	0.920	0.880
Random Forest	0.590	0.454	23%	0.910	0.878
Balanced Random Forest	0.519	0.437	16%	0.890	0.874
Extra Trees	0.553	0.417	25%	0.899	0.862
MLP (128-64)	0.510	0.408	20%	0.838	0.819
Bagging (Decision Tree)	0.578	0.398	31%	0.890	0.855
Logistic Regression	0.470	0.392	17%	0.872	0.854
Linear SVM (calibrated)	0.462	0.388	16%	0.872	0.856
AdaBoost	0.440	0.378	14%	0.873	0.858
Gradient Boosting (sklearn)	0.546	0.373	32%	0.886	0.859
LDA	0.462	0.362	21%	0.853	0.829
Decision Tree	0.412	0.309	25%	0.822	0.758
KNN (k=25, top-60 feats, 30k rows)	0.438	0.297	32%	0.812	0.769
LightGBM (V1 recipe, weighted)	0.273	0.248	9%	0.844	0.828
RBF SVM (top-60 feats, 15k rows)	0.236	0.219	7%	0.839	0.805
Gaussian NB	0.090	0.090	—	0.750	0.751
Dummy (prior)	0.035	0.035	—	0.500	0.500

The ranking of the top models is stable across the two tests, but every score is much higher on the random split, because neighbouring-in-time transactions share cards and patterns.



Same models under the two tests.

6.3 Reading the benchmark

- Dummy baseline PR-AUC equals the fraud rate (0.035). Every real model beats it by a wide margin; Gaussian NB is the weakest real model.
- Boosting and ensembles lead: Voting: XGB + LightGBM + CatBoost (0.488), then Stacking (LR meta): XGB + LightGBM + CatBoost (0.486), then XGBoost (0.480).
- Linear models reach about 0.39; distance-based and kernel models (KNN, RBF-SVM) do poorly, but they were trained on far fewer rows, so they are not a fully fair comparison.
- Stacking with a Logistic Regression meta-learner is not better than plain voting here (0.486 versus 0.488).
- Fit times in the table are wall-clock seconds on a shared 2-CPU machine; some include time when other jobs were running, so use them only as a rough guide.

7 Features, imbalance and tuning

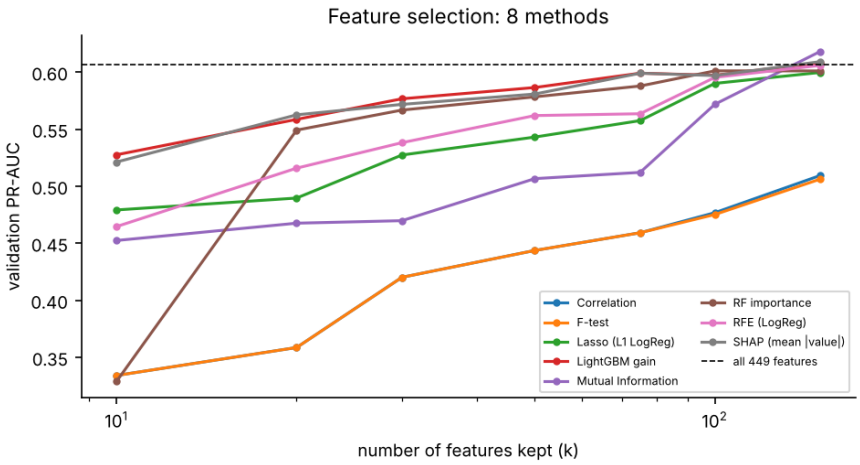
7.1 Feature configurations

Feature set	# features	Val PR	Test PR	Test ROC
F1: V1 seven features	7	0.242	0.149	0.746
F2: raw Gold columns only (no engineered)	432	0.606	0.474	0.884
F3: raw + frequency	441	0.608	0.470	0.884
F4: raw + frequency + 1h rolling (V1 engineering)	444	0.608	0.469	0.884
F5: raw + freq + 1h/24h rolling	447	0.613	0.474	0.889
F6: V2 full (adds card prior-mean amount)	449	0.606	0.476	0.888
F7: V2 full minus time-of-day features	446	0.610	0.469	0.885

LightGBM, default settings, same 100,000-row training sample, time-ordered split. Row F1 is the V1 temporal feature set.

The jump from F1 to F2 is the whole story: V1 temporal mode used 7 columns; the other Gold columns carry the signal. Frequency, 1-hour and 24-hour rolling counts and the card prior-mean amount change test PR-AUC by less than the run-to-run noise (about 0.004). On this dataset, the engineered rolling features are not where the value is.

7.2 Feature selection: 8 methods



Validation PR-AUC against number of features kept, for 8 selection methods (LightGBM, fixed settings).

Method	10	20	30	50	75	100	150
Correlation	0.334	0.359	0.420	0.444	0.459	0.477	0.509
F-test	0.334	0.359	0.420	0.444	0.459	0.475	0.506
Lasso (L1 LogReg)	0.479	0.490	0.527	0.543	0.557	0.590	0.600
LightGBM gain	0.528	0.559	0.577	0.586	0.599	0.597	0.608
Mutual Information	0.452	0.468	0.470	0.507	0.512	0.572	0.618
RF importance	0.329	0.549	0.567	0.578	0.588	0.601	0.601
RFE (LogReg)	0.465	0.516	0.538	0.562	0.564	0.595	0.606
SHAP (mean |value|)	0.521	0.563	0.572	0.581	0.599	0.597	0.609

Validation PR-AUC. Baseline with all 449 features: 0.606 (validation), 0.476 (test).

Decision: keep all 449 features. The best subset (*Mutual Information*, k=150) passed the 1% rule on validation (1.9%), but on the untouched test block it lost 4.3%. We compared 56 candidates on one validation block, so the best of them can look good by chance. We therefore did not adopt it. LightGBM-gain and SHAP subsets of about 150 features match the full set, so a lean deployment option exists: the "tuned + top-150" row in Section 8.

7.3 Class imbalance: SMOTE and others

Strategy	Train rows	Fraud share	Val PR	Test PR	Test ROC	Brier	Val gain
none	100,000	3.5%	0.605	0.480	0.888	0.0236	+0.0%
class weight (full ratio)	100,000	3.5%	0.397	0.261	0.804	0.0314	-34.3%
class weight (sqrt ratio)	100,000	3.5%	0.609	0.471	0.879	0.0243	+0.8%
random undersample 1:4	17,695	20.0%	0.594	0.454	0.882	0.0336	-1.7%
SMOTE (to 1:4)	120,576	20.0%	0.611	0.483	0.889	0.0235	+1.1%
BorderlineSMOTE (to 1:4)	120,576	20.0%	0.608	0.479	0.889	0.0235	+0.6%
SMOTE+Tomek (to 1:4)	119,848	19.8%	0.608	0.478	0.887	0.0236	+0.6%

One seed, applied to the training sample only. SMOTE, Borderline-SMOTE and SMOTE+Tomek interpolate in feature space; coded categories become fractional, a known weakness.

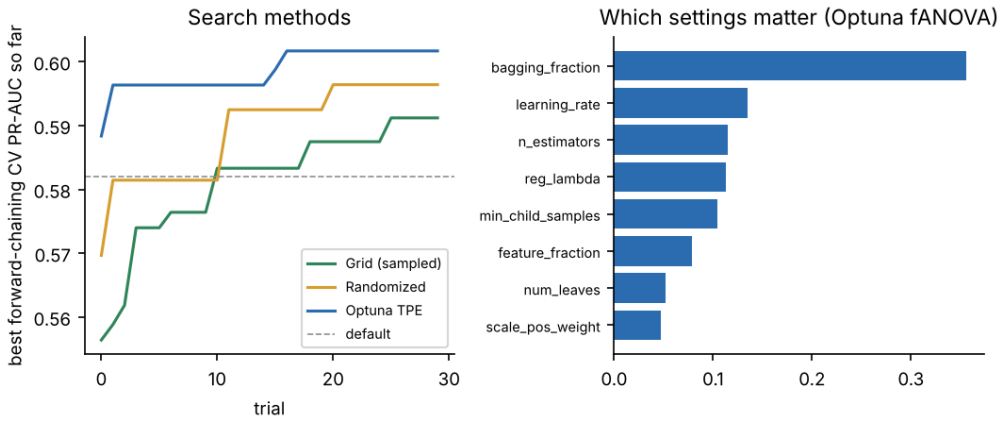
Over three seeds the picture is: no rebalancing 0.6064 ± 0.0044 (validation), SMOTE 0.6095 ± 0.0025 (gain 0.5%; test 1.8%). **Decision: no rebalancing.** The one-seed table above briefly crossed the 1% line (+1.1%), the three-seed check did not. Full-ratio class weighting is clearly harmful (PR-AUC 0.261).

7.4 Hyper-parameter search

Three searches, same budget (30 trials), same objective: mean PR-AUC over three forward-chaining time folds inside the training sample.

Method	CV PR	Val PR	Test PR	Test ROC	Trials
Default	0.582	0.606	0.476	0.888	—
Grid search (sampled)	0.591	0.609	0.475	0.883	30
Randomized search	0.596	0.602	0.469	0.877	30
Optuna (TPE)	0.602	0.620	0.481	0.890	30

Grid search is a fixed-seed sample of the full grid (30 of 23,328 combinations), not an exhaustive grid.



Left: best score found so far in each search. Right: which settings mattered most, from Optuna fANOVA (30 trials only, so a rough guide).

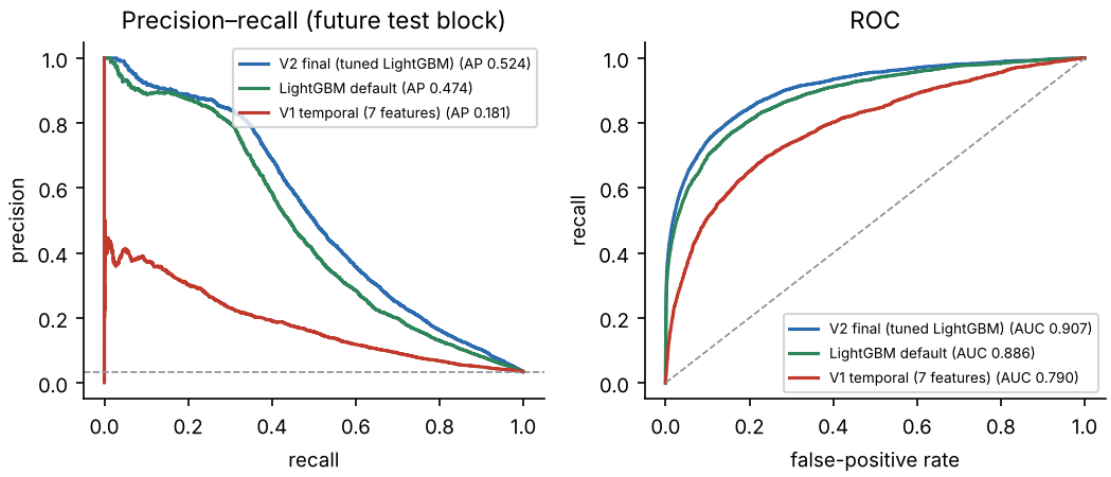
Search method barely matters (the three end within 1.8% of each other on CV). The settings that mattered most were bagging fraction and learning rate; number of leaves is the one that most reliably separated good from poor trials (Spearman 0.75 with CV score). Optuna's best: n_estimators 750, learning_rate 0.0271, num_leaves 152, min_child_samples 41, feature_fraction 0.57, bagging_fraction 0.857, reg_lambda 0.0278, scale_pos_weight 1.07. Hyper-parameter importance from 30 trials is noisy.

8 Final comparison and finance metrics

Candidate	Val PR	Test PR	95% CI (test PR)	Test ROC
Dummy (prior)	0.041	0.035	0.034–0.035	0.500
V1 temporal model (7 features)	—	0.181	0.170–0.190	0.790
LightGBM default	0.608	0.474	0.460–0.489	0.886
Best individual: XGBoost	0.615	0.480	0.465–0.495	0.887
Best ensemble: Voting: XGB + LightGBM + CatBoost	0.621	0.488	0.474–0.502	0.892
LightGBM tuned	0.620	0.481	0.466–0.495	0.890
LightGBM tuned + top-150 features	0.615	0.472	0.458–0.486	0.888
Voting: tuned LightGBM + XGBoost + CatBoost	0.624	0.489	0.475–0.502	0.893

Dummy, baseline models and ensembles trained on the 100,000-row sample. 95% intervals come from 200 bootstrap resamples of the test block.

Rule applied. Incumbent: tuned LightGBM (single model, TreeSHAP-explainable). Best challenger: Voting: tuned LightGBM + XGBoost + CatBoost, 0.71% better on validation. Below 1%, so the incumbent wins. It is then refitted on all 376,469 rows of the fit block (test PR-AUC 0.524; the 100,000-row version scored 0.481). Against LightGBM default, the final model gains 0.050 PR-AUC (95% interval 0.045–0.056).



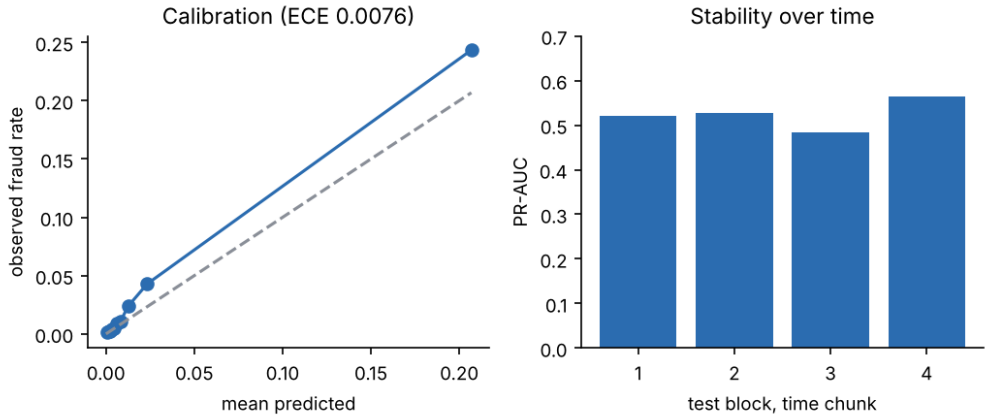
Precision–recall and ROC curves on the future test block.

8.1 Finance and risk metrics (test block)

Metric	Dummy	V1 model	LGBM default	Best ensemble	V2 final
Operating threshold	0.00	0.60	0.18	0.22	0.21
Alert rate (share flagged)	100.0%	11.2%	2.9%	2.3%	2.4%
Precision	0.035	0.155	0.515	0.602	0.619
Recall	1.000	0.505	0.432	0.406	0.434
F1	0.067	0.237	0.469	0.485	0.510
MCC	0.000	0.235	0.454	0.480	0.504
ROC-AUC	0.500	0.790	0.886	0.892	0.907
PR-AUC	0.035	0.181	0.474	0.488	0.524
Gini	0.000	0.579	0.771	0.784	0.813
KS	0.017	0.455	0.613	0.623	0.654
Brier score	0.0334	0.1432	0.0236	0.0233	0.0225
Calibration error (ECE)	0.0008	0.2980	0.0052	0.0050	0.0076
Lift in top 1% of scores	0.8	10.8	24.6	24.8	25.2
Lift in top 5%	1.0	6.4	10.5	10.8	11.5
Recall in top 1% / 5%	0.01 / 0.05	0.11 / 0.32	0.25 / 0.53	0.25 / 0.54	0.25 / 0.57
Fraud *value* caught, top 1% / 5%	0.01 / 0.06	0.04 / 0.18	0.14 / 0.43	0.14 / 0.44	0.14 / 0.53
Recall at 1% / 5% false-positive rate	—	0.15 / 0.36	0.40 / 0.59	0.41 / 0.60	0.44 / 0.64

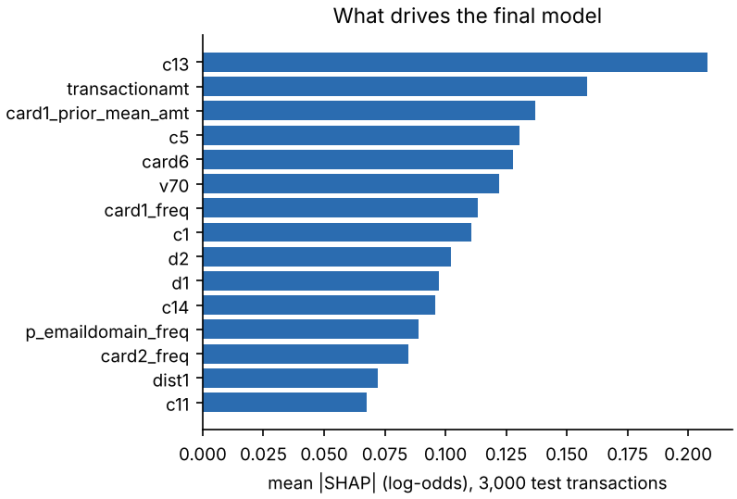
Thresholds: V1 uses its own 0.6; every V2 model uses the F1-optimal threshold from the validation block; Dummy flags everything. Lift = precision in the group divided by the overall fraud rate (3.45%).

Stability over time. The test block was cut into four consecutive time chunks. PR-AUC per chunk: 0.520, 0.528, 0.483, 0.564; ROC-AUC: 0.914, 0.909, 0.897, 0.906. No chunk collapses, so the model does not decay across the 147,635 test transactions. **Drift check (PSI).** Score PSI between validation and test is 0.0011 (common rule of thumb: below 0.1 is stable). Largest feature PSI among the 15 top features: card1_prior_mean_amt (0.087), also below 0.1.



Left: predicted versus observed fraud rate in 10 score groups. Right: PR-AUC in four time chunks of the test block.

8.2 What drives the model (SHAP)



Mean absolute SHAP value of the 15 strongest features on 3,000 test transactions.

The strongest drivers are Vesta count features (C13, C5, C1, C14), the transaction amount, the card's prior mean amount, card type (card6), the card frequency and the D-timedelta features. Vesta does not publish what the C, D and V columns mean, so these explanations are statistically correct but not fully human-readable. Rolling features built in V1 do not appear in the top 15.

9 DGraph-Fin, corrected

9.1 The leak, shown by ablation

Features	ROC-AUC	PR-AUC	Precision@0.5	Recall@0.5
V1 exact (17 raw + degree + timestamp, empty → 0)	0.930	0.560	0.997	0.500
17 raw + timestamp	0.909	0.543	0.996	0.500
17 raw + degree	0.761	0.035	0.149	0.003
17 raw features only	0.726	0.026	0.037	0.001

Same random 70/15/15 split (seed 42), same LightGBM settings; full labelled data (1,225,601 nodes). Adding the timestamp turns a PR-AUC of 0.026 into 0.54.

Published results for this dataset sit in the range of the raw-feature row (DGraph paper: GraphSAGE AUC 0.778, AP 0.043; GADBench: AUPRC about 2–4% for all methods). The V1 numbers (ROC-AUC 0.938, precision 0.99) **must not be used** in any paper or application.

9.2 V2 feature sets (no timestamp)

Feature set	Test ROC-AUC	Test PR-AUC	# features
raw17	0.713	0.025	17
raw17+degree	0.767	0.036	20
all	0.784	0.041	37

`all` = 17 raw + 3 degree features + mean of each neighbour's 17 raw features (the neighbour-aggregation idea that GADBench found strongest for tree models). Degree and neighbour features use the whole graph, so this split is random (the published protocol), not time-ordered.

9.3 25-model benchmark on DGraph-Fin

#	Model	Val PR	Test PR	Test ROC	KS
1	Stacking (LR meta): XGB + LightGBM + CatBoost	0.0434	0.0447	0.793	0.448
2	CatBoost	0.0429	0.0436	0.791	0.442
3	Stacking (LR meta): RF + ExtraTrees + CatBoost	0.0427	0.0433	0.791	0.441
4	Voting: XGB + LightGBM + CatBoost	0.0425	0.0447	0.792	0.453
5	Linear SVM (calibrated)	0.0425	0.0424	0.789	0.441
6	Logistic Regression	0.0424	0.0419	0.789	0.445
7	XGBoost	0.0423	0.0431	0.789	0.443
8	LDA	0.0408	0.0413	0.781	0.423
9	AdaBoost	0.0401	0.0408	0.781	0.421
10	Balanced Random Forest	0.0394	0.0404	0.786	0.438
11	LightGBM (default weights)	0.0388	0.0413	0.784	0.441
12	HistGradientBoosting	0.0388	0.0383	0.780	0.437

Top 12 of 25 by validation PR-AUC (150,000-row training sample, 183,841-row test split with 2,327 frauds). The full table is in the CSV files.

All models land at PR-AUC 0.03–0.045 and the top ten are within a few percent of each other. This matches the base paper (GADBench): on DGraph-Fin every method does poorly. A threshold of 0.5 gives almost no alerts; at the validation-optimal threshold the best F1 is about 0.09. **DGraph-Fin is not ready for production use with these features.**

10 SDG Lab: manual and engine-driven

The old SDG calculator only did arithmetic on numbers typed by the user. The V2 SDG Lab has two tabs.

- **Tab 1, Manual calculator.** The user types volume, fraud rate, average amount, recall and false-alarm rate. Everything is a SCENARIO.
- **Tab 2, Engine-driven.** The fraud engine scores real held-out transactions. Its decisions (fraud / not fraud, plus the real amount) are passed as a payload to the SDG engine, which counts real stopped and leaked money, then adds a queueing model for analyst workload and a Monte Carlo range. A custom payload can also be posted.

Every output carries a tag: **MEASURED** (counted from the payload), **DERIVED** (arithmetic on measured values and stated assumptions) or **SCENARIO** (scaled-up or assumed). Wording is always *aligned with*, never *contributes X%*. No physics-based model applies to fraud, so we use the nearest honest equivalents from operations research: an Erlang-C queue for analyst workload and a Monte Carlo simulation for uncertainty.

10.1 Engine replay on the real test block

Quantity	Value	Tag
Frauds in window / caught	5,100 / 2,215	MEASURED
False alarms (honest flagged)	1,365 (96 per 10,000 honest)	MEASURED
Real fraud value / caught	\$791,044 / \$259,003 (32.7% by value)	MEASURED
Value stopped (90% prevention assumed)	\$233,103	DERIVED
Value leaked	\$557,941	DERIVED
Operating cost (\$2 review, \$1 friction assumed)	\$8,525	DERIVED
Net benefit	\$224,578	DERIVED
Net benefit, Monte Carlo 5th–95th percentile	\$172,022 – \$250,388	SCENARIO
Interest on money held during review (4 h at 5.83%)	\$6	DERIVED

147,635 real held-out transactions, threshold 0.21. The RBI call-money rate (5.83%) comes from the Gold layer; holding honest payments for review costs almost nothing in interest. The real cost of a false alarm is customer friction.

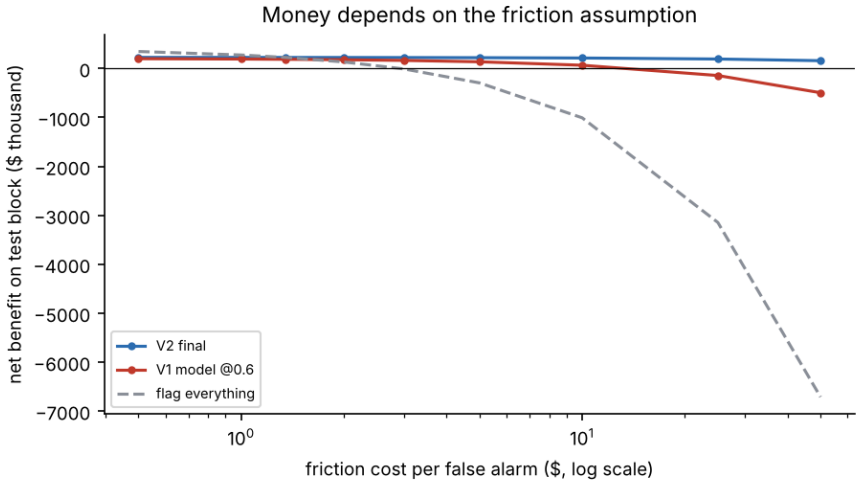
10.2 Scaling up (all SCENARIO)

Per day	1 million transactions	802 million (UPI scale)
Frauds caught per day	15,003	12,032,580
False alarms per day	9,246	7,415,112
Value stopped per day	\$1,578,913	\$1,266,287,957
Value leaked per day	\$3,779,190	\$3,030,910,591
Analyst-hours per day (5 min each)	2,021	1,620,641
Analysts needed for $\leq$10 min mean wait	85	—

Scaled from the test-block rates. The UPI row is arithmetic only: IEEE-CIS is US card data with a larger ticket size, so it must not be read as a UPI forecast. The 802 million per day is an assumed UPI-scale volume, not a value taken from our data.

Micro and macro view. *Micro:* per-transaction economics (stopped value, review cost, friction, interest on held funds). The interest on held funds is tiny; the cost that matters is the friction assumption. *Macro:* scale (1 million to 802 million payments a day), analyst capacity and average ticket (INR 1,600 for UPI, from the Gold NPCI table, against \$135 here). At UPI scale even a 1% false-alarm rate means millions of honest payments flagged every day, so the model alone is not enough: a deployed system would need the alert volume reduced, for example with stronger features and a higher threshold. SDG alignment: 16.4 (illicit financial flows), 8.10 (strengthening financial institutions), 9.c and 1.4 (access to reliable financial services), shown in the Lab as descriptive indicators.

11 Money and the friction assumption



Net benefit on the 147,635-transaction test block (review cost $2, 90% of caught fraud value stopped) for different friction costs per false alarm.

Friction $ / false alarm	V2 model	Flag everything	Better
1	$224,578	$274,134	Flag everything
5	$219,118	-$296,006	Model
10	$212,293	-$1,008,681	Model
25	$191,818	-$3,146,706	Model
50	$157,693	-$6,710,081	Model

Review cost $2 per alert. Fraud value stopped uses real transaction amounts.

What this means. V1 used a $1 friction cost. At that value, blocking every transaction beats any model, which is not a sensible policy. A false alarm costs more than a dollar to a real customer (a declined or delayed payment, support contact, lost trust). V2 therefore reports the model against flag-everything across a range of friction costs and the break-even point (about $1.35 for the final model). The right friction value is a business decision that must come from the bank, not from us.

12 Limitations and honest notes

- IEEE-CIS is US card data in dollars, not UPI. No dataset here is real UPI data. UPI is used for motivation and scale only.
- Benchmark models are fitted on a 100,000-row sample (KNN 30,000, RBF-SVM 15,000). More rows would raise most scores, and the final model uses all 376,469 fit rows.
- Many numbers come from one seed. Where we checked (SMOTE) the run-to-run standard deviation was about 0.004 PR-AUC, so differences smaller than that should not be read as real.
- Vesta's V, C and D columns are engineered by the data owner and we cannot verify that every one of them would be available at scoring time in a live system. This is a property of the public dataset.
- GraphSAGE and the V1 stacked hybrid were not retrained in V2 (no deep-learning stack in the evaluation sandbox). In V1 the graph model gave no measurable forward-time benefit; that conclusion is carried over, not re-tested.
- The Ethereum side experiment is not in the exported Gold files and was not re-run.

- Column M4 is empty because of an ETL defect (Section 2). Fixing it may raise scores; this is not tested.
- The SDG Lab monetary results depend on assumptions (prevention rate, review and friction cost) that are labelled as such. Macro figures are scenario arithmetic.
- Hyper-parameter importance comes from 30 Optuna trials and is noisy. Fit times were measured on a shared 2-CPU machine.
- Multiple comparisons: with 25 models, 56 feature subsets and 90 tuning trials, small validation gains can be luck. The 1% rule, the untouched test block and bootstrap intervals reduce, but do not remove, this risk.

13 Next steps

1. Run `select m4, count(*) from bronze.raw_ieee_cis group by 1`, fix the Silver step, rebuild Gold, and re-run the V2 pipeline.
2. Retrain GraphSAGE on a machine with PyTorch Geometric and test it on the time-ordered split.
3. Agree a friction-cost range with a domain expert, then fix the operating threshold from it.
4. Repeat the main comparisons over several seeds and report intervals for every row.
5. Find or request real UPI-like data before making any claim about UPI performance.

14 Evidence and reproducibility

All steps are scripts with fixed seeds (42 unless stated). Result files are CSV or JSON under `results/`; logs under `logs/`. SHA-256 hashes of the final result files are in `results/final/manifest_sha256.json`.

Script	Purpose
build_combined_gold.py	Combined Gold store, catalogue, unified events
v1_rerun.py, v1_dgraph.py	V1 re-run and DGraph-Fin leak ablation
v2_eda.py	Univariate, bivariate, multivariate analysis
v2_features.py, v2_data.py	Leak-safe time-ordered features and splits
v2_benchmark.py	25-model benchmark (time-ordered, random, DGraph-Fin)
v2_featsel.py, v2_featsets.py	Feature selection (8 methods) and feature configurations
v2_imbalance.py, v2_smote_seeds.py	SMOTE and rebalancing
v2_tune.py	Grid, Randomized, Optuna search
v2_final.py, fin_metrics.py	Final comparison, finance metrics, SDG replay
sdg_engine.py, backend/routers/sdg_v2.py, frontend SdgLab.jsx	SDG Lab engine, API and two-tab page
make_figs.py, build_report.py	Figures and this report